stacking-models 9 Branches 0 Tags Go to file Code

FrancisNdirangu adding presentation and crispdm report 361e7f0 · 1 minute ago 17 Commits

ReadMe.md	final commit	2 days ago
crispdm.pdf	adding presentation and crispdm report	1 minute ago
judge-1377884607_tweet_product_company.csv	first commit	5 days ago
notebook.ipynb	adding presentation and crispdm report	1 minute ago
presentation (1).pdf	adding presentation and crispdm report	1 minute ago
tweet_predictions.csv	added comments,markdown explanations, recommendation...	2 days ago

README

NLP Project: Sentiment Analysis on Tweets about Brands/Products

Group 5 Members

Francis Ndirangu

Brian Kipyegon

Denis Kibor

Ian Riua

Hanifa Chepchirchir

Project Objective This project focuses on developing a sentiment analysis model that predicts whether a tweet expresses a positive, negative, or neutral opinion about a brand or product.

Such a model provides valuable insights for businesses to:

Understand public perception

React proactively to negative feedback

Optimize product development and marketing strategies based on customer sentiment

Dataset Overview Source: CrowdFlower Twitter Dataset

Format: CSV file containing labeled tweets

Key Columns: tweet_text: The actual tweet content (input feature)

emotion: The labeled sentiment (target label), which can be:

Positive emotion

Negative emotion

No emotion toward brand or product

I can't tell

Problem Statement Manually analyzing thousands of tweets to assess customer sentiment is impractical and inefficient. To solve this, we build a machine learning pipeline that can:

Automatically analyze tweets

Predict sentiment in real-time

Help businesses monitor brand reputation and make data-driven decisions

Tools & Libraries Programming Language Python

Libraries Used Data Processing: pandas, numpy

NLP: nltk (stopwords, lemmatization, tokenization), re, emoji

Machine Learning: scikit-learn, xgboost

NLP Techniques We applied several preprocessing steps to clean and prepare raw tweet data:

Noise Removal: URLs, hashtags, mentions, emojis

Tokenization: Splitting text into individual words

Stopword Removal: Filtering out common non-informative words

Lemmatization: Reducing words to their root form (e.g., "running" → "run")

TF-IDF Vectorization: Converting text into numerical features for machine learning models

Machine Learning Models Used We trained and evaluated the following models:

Logistic Regression

Decision Tree Classifier

Random Forest Classifier

Support Vector Machine (SVM)

XGBoost Classifier

Stacking Classifier (Ensemble of multiple models)

Project Workflow

Data Understanding Reviewed dataset structure and identified key features

Analyzed sentiment label distribution

Detected missing and noisy data

Data Cleaning Removed HTML tags, emojis, URLs, and special characters

Dropped rows with missing or unusable tweet content

Preprocessing Applied tokenization, stopword removal, and lemmatization

Transformed clean text into TF-IDF feature vectors

Modeling Performed train-test split

Used Pipeline and GridSearchCV for model optimization

Trained multiple classifiers for comparison

Evaluation Generated classification reports with:

Accuracy

Precision

Recall

F1-score

Exploratory Data Analysis (EDA) Most tweets are short, informal, and contain noisy characters

Sentiment classes are imbalanced, with more neutral and positive tweets than negative ones

Preprocessing improved model performance by standardizing text inputs

Results & Recommendations Key Findings: Ensemble models (like StackingClassifier) provided the highest accuracy

TF-IDF + Logistic Regression gave a strong and interpretable baseline

SVM and XGBoost performed well on non-linear patterns

Future Improvements: Explore deep learning models (e.g., LSTM, BERT) for more accurate context-aware predictions

Use larger and more diverse datasets

Apply sentiment scoring instead of only classification

© 2025 GitHub, Inc. Terms Privacy Security Status Docs Contact Manage cookies Do not share my personal information